

Best Practices for Java Generics

Using **Generics** effectively ensures **type safety, reusability, and maintainability** in Java applications. Below are the key best practices to follow:

1. Use Generics to Ensure Type Safety

- Prevents `ClassCastException` at runtime.
 - Ensures type checking at **compile-time** rather than runtime.
-

2. Prefer Generic Methods Over Overloading

- Reduces redundancy by allowing **a single method** to handle multiple data types.
 - Improves **code reusability** without requiring multiple overloaded methods.
-

3. Use Upper Bounded Wildcards (? extends T) for Read-Only Access

- Allows **reading** elements from a collection without modification.
 - Useful when working with **inherited types** to ensure flexibility.
-

4. Use Lower Bounded Wildcards (? super T) for Write Operations

- Allows **modifying** a collection while maintaining compatibility with **superclasses**.
 - Prevents unintended operations that could introduce type mismatch errors.
-

5. Avoid Using Raw Types (List Instead of List<T>)

- Raw types bypass **type safety**, leading to **unchecked warnings** at compile-time.

- Always use **parameterized types** (`List<String>`, `List<Integer>`) instead of raw `List`.
-

6. Use Bounded Type Parameters for Restriction (<T extends SomeClass>)

- Restricts **type parameters** to a specific class or interface.
 - Ensures **only valid types** can be used with a generic class or method.
-

7. Favor Generic Interfaces for Common Behaviors

- Improves **code reuse** by defining a common behavior for **multiple implementations**.
 - Helps in designing **flexible APIs** that work with different data types.
-

8. Minimize Wildcard Usage in Public APIs

- Use wildcards (`? extends T`, `? super T`) **only when necessary** to improve API flexibility.
 - Avoid wildcards in **method return types**, as it complicates type inference.
-

9. Combine Generics with Functional Interfaces and Streams

- Works well with **Java Streams API** for **processing collections dynamically**.
 - Improves **readability** and **efficiency** in functional-style programming.
-

10. Use Generic Constructors Where Necessary

- Allows creating **type-safe** instances in a **flexible** way.
 - Improves **encapsulation** while maintaining **generic behavior**.
-

11. Avoid Type Erasure Pitfalls

- Remember that **type parameters do not exist at runtime** due to **Type Erasure**.
 - Cannot use **instanceof** with generic type parameters (**T**), as type information is erased.
-

12. Favor Composition Over Inheritance in Generic Hierarchies

- Reduces **complexity** by avoiding deep inheritance chains.
 - Enhances **maintainability** and **flexibility** by composing objects rather than inheriting them.
-

13. Keep Generics Simple and Understandable

- Avoid **overly complex** generic hierarchies.
- Use meaningful type parameter names (**T**, **E**, **K**, **V**) to improve **code readability**.

1. Smart Warehouse Management System

Concepts: Generic Classes, Bounded Type Parameters, Wildcards

Problem Statement:

You are developing a **Smart Warehouse System** that manages different types of items like **Electronics, Groceries, and Furniture**. The system should be able to store and retrieve items dynamically while maintaining type safety.

Hints:

- Create an **abstract class** `WarehouseItem` that all items extend (`Electronics, Groceries, Furniture`).
- Implement a **generic class** `Storage<T extends WarehouseItem>` to store items safely.
- Implement a **wildcard method** to display all items in storage regardless of their type (`List<? extends WarehouseItem>`).

```
package com.smartwarehouse;

import java.util.*;

// Abstract class for Warehouse Items
abstract class WarehouseItem {
    private String name;
    private double price;

    public WarehouseItem(String name, double price) {
        this.name = name;
        this.price = price;
    }

    public String getName() {
        return name;
    }

    public double getPrice() {
        return price;
    }

    @Override
```

```
public String toString() {
    return "Item: " + name + ", Price: " + price;
}
}
```

```
package com.smartwarehouse;

// Subclass for Electronics
class Electronics extends WarehouseItem {
    public Electronics(String name, double price) {
        super(name, price);
    }
}
```

```
package com.smartwarehouse;

class Furniture extends WarehouseItem {
    public Furniture(String name, double price) {
        super(name, price);
    }
}
```

```
package com.smartwarehouse;

class Groceries extends WarehouseItem {
    public Groceries(String name, double price) {
        super(name, price);
    }
}
```

```
package com.smartwarehouse;

class Groceries extends WarehouseItem {
    public Groceries(String name, double price) {
        super(name, price);
    }
}
```

```
}  
}
```

```
package com.smartwarehouse;  
  
public class Main {  
    public static void main(String[] args) {  
        Storage<WarehouseItem> storage = new Storage<>();  
  
        // Adding different types of items  
        storage.addItem(new Electronics("Laptop", 80000));  
        storage.addItem(new Groceries("Rice", 1200));  
        storage.addItem(new Furniture("Chair", 2500));  
  
        // Displaying all items  
        storage.displayItems();  
    }  
}
```

```
package com.smartwarehouse;  
  
import org.junit.jupiter.api.Test;  
import java.util.List;  
  
import static org.junit.jupiter.api.Assertions.*;  
  
class SmartWarehouseTest {  
  
    @Test  
    void testAddAndRetrieveItems() {  
        Storage<WarehouseItem> storage = new Storage<>();  
        storage.addItem(new Electronics("Laptop", 80000));  
        storage.addItem(new Groceries("Rice", 1200));  
        storage.addItem(new Furniture("Chair", 2500));  
  
        List<WarehouseItem> items = storage.getItems();  
        assertEquals(3, items.size());  
        assertTrue(items.get(0) instanceof Electronics);  
        assertEquals("Laptop", items.get(0).getName());  
    }  
}
```

```
@Test
void testDisplayItems() {
    Storage<WarehouseItem> storage = new Storage<>();
    storage.addItem(new Electronics("Phone", 60000));
    storage.addItem(new Groceries("Wheat", 1000));

    storage.displayItems();
    // Visual verification to confirm the output is as expected
}
}
```

2. Dynamic Online Marketplace

Concepts: Type Parameters, Generic Methods, Bounded Type Parameters

Problem Statement:

Build a **generic product catalog** for an online marketplace that supports various product types like **Books, Clothing, and Gadgets**. Each product type has a **specific price range and category**.

Hints:

- Define a **generic class** `Product<T>` where `T` is restricted to a category (`BookCategory`, `ClothingCategory`, etc.).
- Implement a **generic method** to apply discounts dynamically (`<T extends Product> void applyDiscount(T product, double percentage)`).
- Ensure type safety while allowing **multiple product categories** to exist in the same catalog.

```
package com.marketplace;

// Generic class representing a product
public class Product<T> {
    private String name;
    private double price;
    private T category;
}
```

```
public Product(String name, double price, T category) {
    this.name = name;
    this.price = price;
    this.category = category;
}

// Getters and setters
public String getName() {
    return name;
}

public void setName(String name) {
    this.name = name;
}

public double getPrice() {
    return price;
}

public void setPrice(double price) {
    this.price = price;
}

public T getCategory() {
    return category;
}

public void setCategory(T category) {
    this.category = category;
}

@Override
public String toString() {
    return "Product{name='" + name + "', price=" + price + ", category=" +
category + '}';
}
}
```

```
package com.marketplace;

// Class representing the Book category
public class BookCategory {
    private String genre;
```

```
public BookCategory(String genre) {
    this.genre = genre;
}

public String getGenre() {
    return genre;
}

public void setGenre(String genre) {
    this.genre = genre;
}

@Override
public String toString() {
    return "BookCategory{genre='" + genre + "'}";
}
}
```

```
package com.marketplace;

// Class representing the Clothing category
public class ClothingCategory {
    private String size;

    public ClothingCategory(String size) {
        this.size = size;
    }

    public String getSize() {
        return size;
    }

    public void setSize(String size) {
        this.size = size;
    }

    @Override
    public String toString() {
        return "ClothingCategory{size='" + size + "'}";
    }
}
```

```
package com.marketplace;

// Class representing the Gadget category
public class GadgetCategory {
    private String brand;

    public GadgetCategory(String brand) {
        this.brand = brand;
    }

    public String getBrand() {
        return brand;
    }

    public void setBrand(String brand) {
        this.brand = brand;
    }

    @Override
    public String toString() {
        return "GadgetCategory{brand='" + brand + "'}";
    }
}
```

```
package com.marketplace;

// Utility class containing a generic method to apply a discount to a product
public class MarketplaceUtils {

    // Generic method to apply discount to any Product type
    public static <T> void applyDiscount(Product<T> product, double percentage) {
        if (percentage < 0 || percentage > 100) {
            System.out.println("Invalid discount percentage!");
            return;
        }
        double discountAmount = product.getPrice() * (percentage / 100);
        product.setPrice(product.getPrice() - discountAmount);
        System.out.println("Discount applied. New price: " + product.getPrice());
    }
}
```

```
package com.marketplace;

public class Main {
    public static void main(String[] args) {
        // Creating product instances with different categories
        Product<BookCategory> book = new Product<>("Java Programming", 500, new
BookCategory("Technology"));
        Product<ClothingCategory> clothing = new Product<>("T-shirt", 300, new
ClothingCategory("Medium"));
        Product<GadgetCategory> gadget = new Product<>("Smartphone", 20000, new
GadgetCategory("Samsung"));

        // Displaying product details
        System.out.println("Before Discount:");
        System.out.println(book);
        System.out.println(clothing);
        System.out.println(gadget);

        // Applying discounts
        System.out.println("\nApplying Discounts:");
        MarketplaceUtils.applyDiscount(book, 10); // 10% discount on book
        MarketplaceUtils.applyDiscount(clothing, 20); // 20% discount on clothing
        MarketplaceUtils.applyDiscount(gadget, 5); // 5% discount on gadget

        // Displaying product details after discount
        System.out.println("\nAfter Discount:");
        System.out.println(book);
        System.out.println(clothing);
        System.out.println(gadget);
    }
}
```

```
package com.marketplace;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class MarketplaceTest {

    @Test
    void testAddProduct() {
        // Creating a product with BookCategory
    }
}
```

```

        Product<BookCategory> book = new Product<>("Java Programming", 500, new
BookCategory("Technology"));

        // Checking that product attributes are set correctly
        assertEquals("Java Programming", book.getName());
        assertEquals(500, book.getPrice());
        assertEquals("Technology", book.getCategory().getGenre());
    }

    @Test
    void testApplyDiscount() {
        // Creating a product with GadgetCategory
        Product<GadgetCategory> gadget = new Product<>("Smartphone", 20000, new
GadgetCategory("Samsung"));

        // Applying a 10% discount
        MarketplaceUtils.applyDiscount(gadget, 10);

        // Verifying that the price is reduced by 10%
        assertEquals(18000, gadget.getPrice());
    }

    @Test
    void testInvalidDiscount() {
        // Creating a product with ClothingCategory
        Product<ClothingCategory> clothing = new Product<>("T-shirt", 300, new
ClothingCategory("Medium"));

        // Applying an invalid discount (-10%) which should not change the price
        MarketplaceUtils.applyDiscount(clothing, -10);

        // Verifying that the price remains unchanged
        assertEquals(300, clothing.getPrice());
    }

    @Test
    void testToStringMethod() {
        // Creating a product with BookCategory
        Product<BookCategory> book = new Product<>("Clean Code", 600, new
BookCategory("Programming"));

        // Verifying that the toString method returns the expected string
        String expected = "Product{name='Clean Code', price=600.0,
category=BookCategory{genre='Programming'}}";
        assertEquals(expected, book.toString());
    }

```

```
}  
}
```

3. Multi-Level University Course Management System

Concepts: Generic Classes, Wildcards, Bounded Type Parameters

Problem Statement:

Develop a **university course management system** where different departments offer courses with **different evaluation types** (e.g., **Exam-Based**, **Assignment-Based**, **Research-Based**).

Hints:

- Create an **abstract class** `CourseType` (e.g., `ExamCourse`, `AssignmentCourse`, `ResearchCourse`).
- Implement a **generic class** `Course<T extends CourseType>` to manage different courses.
- Use **wildcards** (`List<? extends CourseType>`) to handle **any type of course** dynamically.

```
package com.university.coursemanagement;  
  
// Abstract class representing a general course type  
public abstract class CourseType {  
    private String courseName;  
    private int durationInWeeks;  
  
    public CourseType(String courseName, int durationInWeeks) {  
        this.courseName = courseName;  
        this.durationInWeeks = durationInWeeks;  
    }  
  
    public String getCourseName() {  
        return courseName;  
    }  
}
```

```
public void setCourseName(String courseName) {
    this.courseName = courseName;
}

public int getDurationInWeeks() {
    return durationInWeeks;
}

public void setDurationInWeeks(int durationInWeeks) {
    this.durationInWeeks = durationInWeeks;
}

@Override
public String toString() {
    return "CourseType{courseName='" + courseName + "', durationInWeeks=" +
durationInWeeks + '}';
}
}
```

```
package com.university.coursemanagement;

import java.util.ArrayList;
import java.util.List;

// Generic class to manage different types of courses
public class Course<T extends CourseType> {
    private List<T> courseList = new ArrayList<>();

    // Add a course to the list
    public void addCourse(T course) {
        courseList.add(course);
    }

    // Getter method to retrieve the list of courses
    public List<T> getCourses() {
        return courseList;
    }

    // Display all courses
    public void displayCourses() {
        System.out.println("Courses in the system:");
        for (T course : courseList) {
            System.out.println(course);
        }
    }
}
```

```
}
}
}
```

```
package com.university.coursemanagement;

// Course type that is assignment-based
public class AssignmentCourse extends CourseType {
    private int numberOfAssignments;

    public AssignmentCourse(String courseName, int durationInWeeks, int
numberOfAssignments) {
        super(courseName, durationInWeeks);
        this.numberOfAssignments = numberOfAssignments;
    }

    public int getNumberOfAssignments() {
        return numberOfAssignments;
    }

    public void setNumberOfAssignments(int numberOfAssignments) {
        this.numberOfAssignments = numberOfAssignments;
    }

    @Override
    public String toString() {
        return super.toString() + ", AssignmentCourse{numberOfAssignments=" +
numberOfAssignments + "}";
    }
}
```

```
package com.university.coursemanagement;

// Course type that is exam-based
public class ExamCourse extends CourseType {
    private int totalMarks;

    public ExamCourse(String courseName, int durationInWeeks, int totalMarks) {
        super(courseName, durationInWeeks);
        this.totalMarks = totalMarks;
    }
}
```

```
public int getTotalMarks() {
    return totalMarks;
}

public void setTotalMarks(int totalMarks) {
    this.totalMarks = totalMarks;
}

@Override
public String toString() {
    return super.toString() + ", ExamCourse{totalMarks=" + totalMarks + "}";
}
}
```

```
package com.university.coursemanagement;

// Course type that is research-based
public class ResearchCourse extends CourseType {
    private String researchTopic;

    public ResearchCourse(String courseName, int durationInWeeks, String
researchTopic) {
        super(courseName, durationInWeeks);
        this.researchTopic = researchTopic;
    }

    public String getResearchTopic() {
        return researchTopic;
    }

    public void setResearchTopic(String researchTopic) {
        this.researchTopic = researchTopic;
    }

    @Override
    public String toString() {
        return super.toString() + ", ResearchCourse{researchTopic='" + researchTopic
+ "'}";
    }
}
```

```
package com.university.coursemanagement;

public class Main {
    public static void main(String[] args) {
        // Create instances of different course types
        ExamCourse examCourse = new ExamCourse("Mathematics", 12, 100);
        AssignmentCourse assignmentCourse = new AssignmentCourse("Data Structures",
10, 5);
        ResearchCourse researchCourse = new ResearchCourse("Artificial
Intelligence", 16, "Machine Learning");

        // Create a Course manager for exam-based courses
        Course<ExamCourse> examCourseManager = new Course<>();
        examCourseManager.addCourse(examCourse);

        // Create a Course manager for assignment-based courses
        Course<AssignmentCourse> assignmentCourseManager = new Course<>();
        assignmentCourseManager.addCourse(assignmentCourse);

        // Create a Course manager for research-based courses
        Course<ResearchCourse> researchCourseManager = new Course<>();
        researchCourseManager.addCourse(researchCourse);

        // Display all courses
        System.out.println("Exam Courses:");
        examCourseManager.displayCourses();

        System.out.println("\nAssignment Courses:");
        assignmentCourseManager.displayCourses();

        System.out.println("\nResearch Courses:");
        researchCourseManager.displayCourses();
    }
}
```

```
package com.coursemanagement;

import com.university.coursemanagement.AssignmentCourse;
import com.university.coursemanagement.Course;
import com.university.coursemanagement.ExamCourse;
import com.university.coursemanagement.ResearchCourse;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;
```

```
import java.util.List;

class CourseManagementTest {

    @Test
    void testAddCourse() {
        // Creating a Course manager for ExamCourse
        Course<ExamCourse> examCourseManager = new Course<>();
        ExamCourse examCourse = new ExamCourse("Mathematics", 12, 100);

        // Adding the course to the manager
        examCourseManager.addCourse(examCourse);

        // Verifying that the course was added
        List<ExamCourse> courses = examCourseManager.getCourses();
        assertEquals(1, courses.size());
        assertEquals("Mathematics", courses.get(0).getCourseName());
        assertEquals(100, courses.get(0).getTotalMarks());
    }

    @Test
    void testDisplayCourses() {
        // Creating a Course manager and adding multiple courses
        Course<AssignmentCourse> assignmentCourseManager = new Course<>();
        assignmentCourseManager.addCourse(new AssignmentCourse("Data Structures",
10, 5));
        assignmentCourseManager.addCourse(new AssignmentCourse("Algorithms", 8, 4));

        // Verifying that the courses are added correctly
        List<AssignmentCourse> courses = assignmentCourseManager.getCourses();
        assertEquals(2, courses.size());
        assertEquals("Data Structures", courses.get(0).getCourseName());
        assertEquals(5, courses.get(0).getNumberOfAssignments());
    }

    @Test
    void testResearchCourseAttributes() {
        // Creating a ResearchCourse and verifying its attributes
        ResearchCourse researchCourse = new ResearchCourse("Artificial
Intelligence", 16, "Machine Learning");

        assertEquals("Artificial Intelligence", researchCourse.getCourseName());
        assertEquals(16, researchCourse.getDurationInWeeks());
        assertEquals("Machine Learning", researchCourse.getResearchTopic());
    }
}
```

```
@Test
void testToStringMethod() {
    // Creating an ExamCourse and checking the toString output
    ExamCourse examCourse = new ExamCourse("Physics", 14, 200);
    String expected = "CourseType{courseName='Physics', durationInWeeks=14},
ExamCourse{totalMarks=200}";

    assertEquals(expected, examCourse.toString());
}
}
```

4. Personalized Meal Plan Generator

Concepts: Generic Methods, Type Parameters, Bounded Type Parameters

Problem Statement:

Design a **Personalized Meal Plan Generator** where users can choose **different meal categories** like **Vegetarian, Vegan, Keto, or High-Protein**. The system should ensure **only valid meal plans** are generated.

Hints:

- Define an **interface MealPlan** with subtypes (**VegetarianMeal, VeganMeal**, etc.).
- Implement a **generic class Meal<T extends MealPlan>** to handle different meal plans.
- Use a **generic method** to validate and generate a personalized meal plan dynamically.

```
package com.mealplan.genetator;

// Interface representing a general meal plan
public interface MealPlan {
    // Method to display meal details, to be implemented by each specific meal type
    void showMealDetails();
}
```

```
package com.mealplan.genetator;

// Generic class to manage different types of meal plans
// T must extend MealPlan, ensuring only valid meal types are used
public class Meal<T extends MealPlan> {
    private T mealPlan; // Meal plan of a specific type

    // Constructor to initialize the meal plan
    public Meal(T mealPlan) {
        this.mealPlan = mealPlan;
    }

    // Method to display the details of the stored meal plan
    public void displayMeal() {
        mealPlan.showMealDetails();
    }
}
```

```
package com.mealplan.genetator;

// Class representing a vegan meal plan
public class VeganMeal implements MealPlan {
    private String smoothie; // Smoothie for the vegan meal

    // Constructor to initialize the smoothie
    public VeganMeal(String smoothie) {
        this.smoothie = smoothie;
    }

    // Implementing the showMealDetails method from MealPlan interface
    // This method displays details about the vegan meal
    @Override
    public void showMealDetails() {
        System.out.println("Vegan Meal - Smoothie: " + smoothie);
    }
}
```

```
package com.mealplan.genetator;

// Class representing a vegetarian meal plan
```

```
public class VegetarianMeal implements MealPlan {
    private String mainDish; // Main dish for the vegetarian meal

    // Constructor to initialize the main dish
    public VegetarianMeal(String mainDish) {
        this.mainDish = mainDish;
    }

    // Implementing the showMealDetails method from MealPlan interface
    // This method displays details about the vegetarian meal
    @Override
    public void showMealDetails() {
        System.out.println("Vegetarian Meal - Main Dish: " + mainDish);
    }
}
```

```
package com.mealplan.genetator;

// Class representing a keto meal plan
public class KetoMeal implements MealPlan {
    private String proteinSource; // Protein source for the keto meal

    // Constructor to initialize the protein source
    public KetoMeal(String proteinSource) {
        this.proteinSource = proteinSource;
    }

    // Implementing the showMealDetails method from MealPlan interface
    // This method displays details about the keto meal
    @Override
    public void showMealDetails() {
        System.out.println("Keto Meal - Protein Source: " + proteinSource);
    }
}
```

```
package com.mealplan.genetator;

// Main class to generate and manage personalized meal plans
public class Main {
```

```
// Generic method to generate a meal plan
// T must extend MealPlan, ensuring only valid meal types can be passed
public static <T extends MealPlan> void generateMealPlan(T mealPlan) {
    System.out.println("Generating Meal Plan...");
    mealPlan.showMealDetails(); // Display the meal details
    System.out.println("Meal Plan Generated Successfully!\n");
}

// Main method to demonstrate meal plan generation and display
public static void main(String[] args) {
    // Creating instances of different meal plans
    VegetarianMeal vegetarianMeal = new VegetarianMeal("Paneer Curry");
    VeganMeal veganMeal = new VeganMeal("Green Smoothie");
    KetoMeal ketoMeal = new KetoMeal("Grilled Chicken");

    // Generating meal plans using the generic method
    generateMealPlan(vegetarianMeal);
    generateMealPlan(veganMeal);
    generateMealPlan(ketoMeal);

    // Using the generic class Meal<T> to display each meal plan
    System.out.println("Displaying Meal Plans:");
    Meal<VegetarianMeal> vegMeal = new Meal<>(vegetarianMeal);
    vegMeal.displayMeal();

    Meal<VeganMeal> vegan = new Meal<>(veganMeal);
    vegan.displayMeal();

    Meal<KetoMeal> keto = new Meal<>(ketoMeal);
    keto.displayMeal();
}
}
```

```
package com.mealplan.generator;

import com.mealplan.genetator.KetoMeal;
import com.mealplan.genetator.Main;
import com.mealplan.genetator.VeganMeal;
import com.mealplan.genetator.VegetarianMeal;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class MealPlanGeneratorTest {
```

```

@Test
void testVegetarianMealDetails() {
    // Creating a VegetarianMeal instance
    VegetarianMeal vegetarianMeal = new VegetarianMeal("Paneer Curry");

    // Verifying the meal details are set correctly
    assertNotNull(vegetarianMeal);
}

@Test
void testVeganMealDetails() {
    // Creating a VeganMeal instance
    VeganMeal veganMeal = new VeganMeal("Green Smoothie");

    // Verifying the meal details are set correctly
    assertNotNull(veganMeal);
}

@Test
void testKetoMealDetails() {
    // Creating a KetoMeal instance
    KetoMeal ketoMeal = new KetoMeal("Grilled Chicken");

    // Verifying the meal details are set correctly
    assertNotNull(ketoMeal);
}

@Test
void testMealDisplay() {
    // Creating instances of different meal plans
    VegetarianMeal vegetarianMeal = new VegetarianMeal("Paneer Curry");
    VeganMeal veganMeal = new VeganMeal("Green Smoothie");
    KetoMeal ketoMeal = new KetoMeal("Grilled Chicken");

    // Verifying that each meal displays without errors
    assertDoesNotThrow(vegetarianMeal::showMealDetails);
    assertDoesNotThrow(veganMeal::showMealDetails);
    assertDoesNotThrow(ketoMeal::showMealDetails);
}

@Test
void testGenerateMealPlan() {
    // Creating a VeganMeal instance
    VeganMeal veganMeal = new VeganMeal("Avocado Smoothie");

```

```
// Verifying the generateMealPlan method runs without exceptions
assertDoesNotThrow(() -> Main.generateMealPlan(veganMeal));
}
}
```

5. AI-Driven Resume Screening System

Concepts: Generic Classes, Generic Methods, Bounded Type Parameters, Wildcards

Problem Statement:

Develop an **AI-Driven Resume Screening System** that can process resumes for **different job roles** like **Software Engineer**, **Data Scientist**, and **Product Manager** while ensuring type safety.

Hints:

- Create an **abstract class** `JobRole` (`SoftwareEngineer`, `DataScientist`, `ProductManager`).
- Implement a **generic class** `Resume<T extends JobRole>` to process resumes dynamically.
- Use a **wildcard method** (`List<? extends JobRole>`) to handle multiple job roles in the screening pipeline.

```
package com.ai.resumescreening;

// Abstract class representing a generic job role
public abstract class JobRole {
    private String jobTitle; // Title of the job role

    // Constructor to initialize the job title
}
```

```
public JobRole(String jobTitle) {
    this.jobTitle = jobTitle;
}

// Method to get the job title
public String getJobTitle() {
    return jobTitle;
}

// Abstract method to display role-specific details
public abstract void showRoleDetails();
}
```

```
package com.ai.resumescreening;

// Generic class to represent a resume for a specific job role
// T must extend JobRole, ensuring only valid job roles are processed
public class Resume<T extends JobRole> {
    private T jobRole; // Job role associated with this resume

    // Constructor to initialize the job role
    public Resume(T jobRole) {
        this.jobRole = jobRole;
    }

    // Method to display details of the associated job role
    public void displayJobDetails() {
        jobRole.showRoleDetails();
    }
}
```

```
package com.ai.resumescreening;

// Class representing the Data Scientist job role
public class DataScientist extends JobRole {
    private String toolsAndTechniques; // Tools and techniques required for this role

    // Constructor to initialize the job title and tools
    public DataScientist(String jobTitle, String toolsAndTechniques) {
        super(jobTitle);
        this.toolsAndTechniques = toolsAndTechniques;
    }
}
```

```

    }

    // Implementing the showRoleDetails method from JobRole
    @Override
    public void showRoleDetails() {
        System.out.println("Data Scientist Role - Job Title: " + getJobTitle());
        System.out.println("Required Tools and Techniques: " + toolsAndTechniques);
    }
}

```

```

package com.ai.resumescreening;

// Class representing the Product Manager job role
public class ProductManager extends JobRole {
    private String managementSkills; // Management skills required for this role

    // Constructor to initialize the job title and skills
    public ProductManager(String jobTitle, String managementSkills) {
        super(jobTitle);
        this.managementSkills = managementSkills;
    }

    // Implementing the showRoleDetails method from JobRole
    @Override
    public void showRoleDetails() {
        System.out.println("Product Manager Role - Job Title: " + getJobTitle());
        System.out.println("Required Management Skills: " + managementSkills);
    }
}

```

```

package com.ai.resumescreening;

// Class representing the Software Engineer job role
public class SoftwareEngineer extends JobRole {
    private String programmingLanguages; // Programming languages required for this role

    // Constructor to initialize the job title and programming languages
    public SoftwareEngineer(String jobTitle, String programmingLanguages) {
        super(jobTitle);
    }
}

```

```

        this.programmingLanguages = programmingLanguages;
    }

    // Implementing the showRoleDetails method from JobRole
    @Override
    public void showRoleDetails() {
        System.out.println("Software Engineer Role - Job Title: " + getJobTitle());
        System.out.println("Required Programming Languages: " +
programmingLanguages);
    }
}

```

```

package com.ai.resumescreening;

import java.util.List;

public class ResumeUtils {
    // Method to process multiple resumes using wildcards
    // Accepts a list of resumes for any type of job role (List<? extends JobRole>)
    public static void processResumes(List<? extends JobRole> resumes) {
        System.out.println("Processing Resumes...");
        for (JobRole role : resumes) {
            role.showRoleDetails();
            System.out.println("----");
        }
        System.out.println("Resume Processing Completed!\n");
    }
}

```

```

package com.ai.resumescreening;

import java.util.List;

public class Main {
    // Main method to demonstrate resume screening
    public static void main(String[] args) {
        // Creating individual job roles
        SoftwareEngineer seRole = new SoftwareEngineer("Backend Developer", "Java,
Spring Boot");
        DataScientist dsRole = new DataScientist("Data Analyst", "Python, SQL,
Machine Learning");
    }
}

```

```

        ProductManager pmRole = new ProductManager("Project Lead", "Agile, Scrum,
Leadership");

        // Creating individual resumes for the job roles
        Resume<SoftwareEngineer> seResume = new Resume<>(seRole);
        Resume<DataScientist> dsResume = new Resume<>(dsRole);
        Resume<ProductManager> pmResume = new Resume<>(pmRole);

        // Displaying details of individual resumes
        System.out.println("Displaying Individual Resumes:");
        seResume.displayJobDetails();
        System.out.println();

        dsResume.displayJobDetails();
        System.out.println();

        pmResume.displayJobDetails();
        System.out.println();

        // Processing multiple resumes in a list using wildcards
        System.out.println("Processing Multiple Resumes:");
        ResumeUtils.processResumes(List.of(seRole, dsRole, pmRole));
    }
}

```

```

package com.ai.resumescreening;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;
import java.util.List;

class ResumeScreeningTest {

    @Test
    void testSoftwareEngineerRoleDetails() {
        // Creating a SoftwareEngineer instance
        SoftwareEngineer seRole = new SoftwareEngineer("Backend Developer", "Java,
Spring Boot");

        // Verifying that job title and programming languages are set correctly
        assertEquals("Backend Developer", seRole.getJobTitle());
    }
}

```

```

@Test
void testDataScientistRoleDetails() {
    // Creating a DataScientist instance
    DataScientist dsRole = new DataScientist("Data Analyst", "Python, SQL,
Machine Learning");

    // Verifying that job title and tools are set correctly
    assertEquals("Data Analyst", dsRole.getJobTitle());
}

@Test
void testProductManagerRoleDetails() {
    // Creating a ProductManager instance
    ProductManager pmRole = new ProductManager("Project Lead", "Agile, Scrum,
Leadership");

    // Verifying that job title and management skills are set correctly
    assertEquals("Project Lead", pmRole.getJobTitle());
}

@Test
void testResumeDisplay() {
    // Creating a resume for a DataScientist
    DataScientist dsRole = new DataScientist("Data Scientist", "Python, R,
SQL");
    Resume<DataScientist> dsResume = new Resume<>(dsRole);

    // Verifying that resume displays without throwing exceptions
    assertDoesNotThrow(dsResume::displayJobDetails);
}

@Test
void testProcessMultipleResumes() {
    // Creating instances of multiple job roles
    SoftwareEngineer seRole = new SoftwareEngineer("Backend Developer", "Java,
Spring Boot");
    DataScientist dsRole = new DataScientist("Data Analyst", "Python, SQL");
    ProductManager pmRole = new ProductManager("Project Lead", "Agile, Scrum");

    // Processing multiple resumes and verifying no exceptions occur
    List<JobRole> roles = List.of(seRole, dsRole, pmRole);
    assertDoesNotThrow(() -> ResumeUtils.processResumes(roles));
}
}

```

CodInClub

Powered By 
BridgeLabz